

Blueprint 1 — Kubernetes manifests explained

2026-09-19 · @Someone

Overview

The 12 files under `blueprint1/manifests/` describe every Kubernetes object in the stack. You never apply them by hand; each phase script fills them in from `config.env` and applies them.

Templating. The files contain placeholders like `${APP_IMAGE}` or `${DECODE_GPUMEM}`. `scripts/lib.sh` runs them through `envsubst`, which swaps in only the variables defined in `config.env` (plus a few set by the scripts, such as `POOL` or `ROUTER_DECODE_URLS`). Anything else containing `$` is left alone.

Namespaces. Objects are grouped into five namespaces: `gpu-operator` and `hami-system` (installed by Helm, no files here), `observability`, `kv-tier` (Mooncake) and `inference` (vLLM, routers, gateway, bench client).

File	Namespace	Applied by	Creates
<code>cluster/runtimeclass-nvidia.yaml</code>	cluster-wide	<code>01_install_k3s_gpu.sh</code>	RuntimeClass
<code>cluster/hami-canary.yaml</code>	default	<code>02_install_hami.sh</code>	Pod (temporary)
<code>observability/podmonitors.yaml</code>	inference, kv-tier	<code>03_install_observability.sh</code>	3 PodMonitors
<code>observability/grafana-dashboard.json</code>	observability	<code>03_install_observability.sh</code>	ConfigMap (dashboard)
<code>kv-tier/mooncake.yaml</code>	kv-tier	<code>04_deploy_mooncake.sh</code>	PV, PVC, 2 Deployments, Service
<code>inference/model-download-job.yaml</code>	inference	<code>05_deploy_vllm.sh</code>	Job

File	Namespace	Applied by	Creates
<code>inference/vllm-statefulset.yaml</code>	inference	<code>05_deploy_vllm.sh</code> , <code>set_config.sh</code>	Headless Service + StatefulSet (once per pool)
<code>router/router-service.yaml</code>	inference	<code>06_deploy_router.sh</code>	Service
<code>router/sglang-router.yaml</code>	inference	<code>06_deploy_router.sh</code> (config B)	Deployment
<code>router/kv-router.yaml</code>	inference	<code>06_deploy_router.sh</code> (config C)	Deployment
<code>gateway/litellm.yaml</code>	inference	<code>07_deploy_gateway.sh</code>	Secret, ConfigMap, Deployment, NodePort Service
<code>bench/bench-client.yaml</code>	inference	<code>08_run_benchmarks.sh</code>	Deployment

How a request travels through these objects in config C:

```

flowchart LR
    C[Client] --> G[litellm Service<br/>NodePort 30400]
    G --> R[router Service :8080]
    R --> K[kv-router pod]
    K -->|1: prefill copy| P[vllm-prefill-0/1]
    K -->|2: real request| D[vllm-decode-0/1/2]
    P --> M[(mooncake-store)]
    M --> D

```

The numbered arrows are the two hops `kv-router` makes for every request; the store in the middle is how the prefill pod's work reaches the decode pod.

Cluster architecture

The whole cluster is one k3s node: five namespaces of pods sharing one H100 through HAMi, with the node's NVMe disk holding the model, caches and results.

```

flowchart TB
    U[Laptop] -->|NodePort 30400| LL
    subgraph NODE[Lambda node: k3s, 1x H100 80 GB]
        subgraph INF[ns: inference]
            LL[litellm<br/>gateway + auth] --> RT[router Service<br/>kv-router or sgl-router]
            RT -->|1: prefill| VP[vllm-prefill x2<br/>16 GB / 25% each]
            RT -->|2: decode| VD[vllm-decode x3<br/>14 GB / 15% each]
            BC[bench-client] --> RT
        end
        end
        subgraph KV[ns: kv-tier]
            MM[mooncake-master<br/>rpc 50051, meta 8080]
            MS[mooncake-store<br/>32 GB RAM]
        end
        end
        subgraph OBS[ns: observability]
            PR[Prometheus] --> GF[Grafana]
        end
        end
        subgraph SYS[ns: hami-system + gpu-operator]
            HM[HAMi scheduler<br/>+ device plugin]
            DC[DCGM exporter]
        end
        end
        GPU[(H100 GPU)]
        DISK[(NVMe: hf-cache,<br/>mooncake-storage, bench)]
    end
    VP & VD <-->|LMCache| MS
    MS --- MM
    HM -.slices.-> GPU
    VP & VD -.HAMi slice.-> GPU
    MS -.spill.-> DISK
    PR -.scrape.-> VD & DC & MM

```

Solid arrows are request and KV-cache traffic; dotted arrows are GPU slicing, disk spill and metrics scraping. In config B the prefill pods and the LMCache links are absent, and in config A a single `vllm-single` pod replaces both pools with no router in front.

Cluster: runtime class and HAMi canary

These two small files make the GPU usable from containers and prove it can be shared.

```
cluster/runtimeclass-nvidia.yaml
```

A **RuntimeClass** tells Kubernetes which low-level container runtime to use for a pod. This one is named `nvidia` and points at the `nvidia` handler, which is NVIDIA's container runtime that k3s's containerd picked up from the host.

- Pods that ask for `runtimeClassName: nvidia` get the GPU driver libraries and device files mounted in.
- k3s usually creates this object itself when it detects the NVIDIA runtime. The phase 1 script applies this file only if it's missing.
- k3s is also started with `--default-runtime=nvidia`, so in practice every pod uses it. HAMi needs that.

```
cluster/hami-canary.yaml
```

A throwaway **Pod** that checks HAMi is really slicing the GPU. It runs once, prints what it sees, and is deleted.

Field	Value	Why
<code>schedulerName</code>	<code>hami-scheduler</code>	HAMi's scheduler places the pod and records its slice; the default scheduler knows nothing about GPU memory
<code>nvidia.com/gpu</code>	1	Asks for one (virtual) GPU
<code>nvidia.com/gpumem</code>	8192	Caps GPU memory at 8 GB (in MB)
<code>nvidia.com/gpucore</code>	20	Caps compute at 20% of the GPU's SMs
<code>restartPolicy</code>	<code>Never</code>	Run once, don't restart
<code>command</code>	<code>nvidia-smi --query-gpu=name,memory.used,memory.total</code>	Prints the memory the container can see

If HAMi works, `nvidia-smi` inside the pod reports **8192 MiB total** even though the H100 has 80 GB. The vLLM pods rely on the same trick: each thinks it has a small GPU of its own.

Observability: PodMonitors and the Grafana dashboard

These files tell Prometheus what to scrape and give Grafana a ready-made dashboard. Prometheus and Grafana themselves come from the `kube-prometheus-stack` Helm chart, which has no file here.

`observability/podmonitors.yaml`

A **PodMonitor** is a custom resource from the Prometheus Operator. It says: "find pods with these labels and scrape this named port every N seconds". Pod-level scraping keeps each replica's numbers separate, so you can see `vllm-decode-0` and `vllm-decode-1`

side by side.

PodMonitor	Namespace	Selects pods labelled	Port name	Scrapes
vllm	inference	app: vllm	http (8000)	Every vLLM pod: TTFT, ITL, tokens/s, KV usage, prefix-cache hits, preemptions
routers	inference	tier: router	metrics	kv-router (:9100) or sgl-router (:29000): requests per worker, prefix-match ratio
mooncake-master	kv-tier	app: mooncake, role: master	metrics (9003)	Mooncake master: store operations

All three use `interval: 15s` and path `/metrics`. Prometheus picks them up because the chart is installed with `podMonitorSelectorNilUsesHelmValues=false`, meaning "watch all PodMonitors, not just the chart's own".

observability/grafana-dashboard.json

A Grafana dashboard definition, not Kubernetes YAML. The phase 3 script wraps it in a **ConfigMap** labelled `grafana_dashboard=1`. Grafana runs a sidecar container that watches for that label and loads the dashboard automatically, so it appears as "Blueprint 1 — Inference stack" with no manual import.

It has 12 panels:

- TTFT p95 and inter-token latency p95, per pod
- Generation and prompt throughput (tokens/s)
- HBM prefix-cache hit rate and KV-cache usage
- Running vs waiting requests, and preemptions per second
- GPU utilisation and HBM used (from DCGM)
- Router requests per worker, router prefix-match ratio, and the P/D prefill-hop latency

Some queries use `or` between two metric names because vLLM has renamed a few metrics over time (for example `kv_cache_usage_perc` vs the older `gpu_cache_usage_perc`).

KV tier: `kv-tier/mooncake.yaml`

This one file builds the shared KV-cache store: a 200 GB NVMe volume, a Mooncake master that keeps track of where every KV chunk lives, and a store node that holds the chunks in 32 GB of RAM. None of these pods touch the GPU.

It holds five objects:

#	Kind	Name	Role
1	PersistentVolume	<code>mooncake-nvme</code>	200 Gi of disk at <code>/home/ubuntu/mooncake-storage</code> on the node
2	PersistentVolumeClaim	<code>mooncake-nvme</code>	The store pod's claim on that volume
3	Deployment	<code>mooncake-master</code>	Directory service: which chunk is where
4	Service	<code>mooncake-master</code>	Stable DNS name + ports for the master
5	Deployment	<code>mooncake-store</code>	Holds the actual KV data

PersistentVolume + PersistentVolumeClaim

A **PersistentVolume (PV)** is a piece of storage the cluster knows about; a **PersistentVolumeClaim (PVC)** is a pod's request for it. Here the PV is a `hostPath`: a plain directory on the node's local NVMe, created if missing (`DirectoryOrCreate`).

- `storageClassName: local-nvme` and `volumeName: mooncake-nvme` bind the claim to exactly this volume, rather than letting k3s's default storage class create a new one.
- `persistentVolumeReclaimPolicy: Retain` keeps the files if the claim is deleted.

`mooncake-master` Deployment

The master doesn't store KV data. It runs `mooncake_master`, which does two jobs:

- **RPC on port 50051:** clients ask it "where can I put this chunk?" and "where is chunk X?".

- **HTTP metadata server on port 8080** (`--enable_http_metadata_server=true`): each participant registers its network address here so others can connect to it directly.

Other settings:

- `--eviction_high_watermark_ratio=0.90` : start evicting old chunks when the pool is 90% full.
- If `MOONCAKE_ENABLE_OFFLOAD=true` in `config.env`, it adds `--enable_offload=true -offload_on_evict=true`, so evicted chunks spill to NVMe instead of being dropped.
- `NVIDIA_VISIBLE_DEVICES=void` hides the GPU. Every pod gets the NVIDIA runtime by default, so this stops a CPU-only pod from accidentally creating a GPU context and using HBM that HAMi isn't tracking.
- A `readinessProbe` on TCP 50051 marks it ready once the RPC port accepts connections.
- It exposes port 9003 for Prometheus.

mooncake-master Service

A normal ClusterIP Service, so everything can reach the master at `mooncake-master.kv-tier.svc.cluster.local` on ports 50051 (rpc), 8080 (metadata) and 9003 (metrics).

mooncake-store Deployment

This pod contributes the memory. It runs `files/mooncake_store_node.py` (mounted from the `mooncake-store-node` ConfigMap), which:

1. Joins the Mooncake cluster, using its own pod IP (`POD_IP`, taken from the downward API) as its address.
2. Registers a 32 GB RAM segment (`MOONCAKE_STORE_GB`) that the master can hand out to clients.
3. Serves `GET /health` on port 8081 for its readiness probe, then waits until it's stopped.

Other details:

- `MOONCAKE_OFFLOAD_FILE_STORAGE_PATH=/data` points spills at the NVMe volume mounted at `/data`.
- It requests 36 Gi of memory with a 40 Gi limit: 32 GB for the segment plus headroom.
- `strategy: Recreate` stops the old pod before starting a new one, so two store pods never hold the same volume.

The vLLM pods reach this store through LMCache, using the master's address. They never talk to the store pod by name.

Inference: model download and the vLLM pools

Two files run the model: a Job downloads the weights once, and one StatefulSet template becomes every vLLM pool in all three configs.

`inference/model-download-job.yaml`

A **Job** runs a pod to completion, then stops. This one calls `huggingface_hub.snapshot_download` for `Qwen/Qwen2.5-7B-Instruct` and saves the files to `/home/ubuntu/hf-cache` on the node (a `hostPath` mount).

- `allow_patterns` limits the download to weights, configs and tokenizer files, about 15 GB.
- `HF_TOKEN` comes from the `hf-token` Secret; `optional: true` lets it run without one.
- `backoffLimit: 3` retries up to three times if the download fails.

Without this Job, five vLLM pods would start at once and all try to download the same 15 GB into the same folder. Instead they start with `HF_HUB_OFFLINE=1` and read the cached copy.

`inference/vllm-statefulset.yaml`

This is a **template for one pool**. The scripts render it once per pool, filling in `POOL`, `ROLE`, `REPLICAS`, the slice size and the KV settings:

Pool	Config	Replicas	Slice (HBM / compute)	Prefix caching	KV tier (LMCache → Mooncake)
<code>vllm-single</code>	A	1	14 GB / 15%	off	no
<code>vllm-decode</code>	B	3	14 GB / 15% each	on	no
<code>vllm-prefill</code>	C	2	16 GB / 25% each	on	yes
<code>vllm-decode</code>	C	3	14 GB / 15% each	on	yes

Each render produces two objects.

Headless Service (`clusterIP: None`). A normal Service hides pods behind one virtual IP and spreads traffic across them. A headless Service instead gives every pod its own DNS name, such as `vllm-decode-0.vllm-decode.inference.svc.cluster.local`. The routers need this: a cache-aware router must pick a *specific* pod, and a normal Service would reshuffle requests underneath it. `publishNotReadyAddresses: true` makes those names resolvable while pods are still starting.

StatefulSet. Like a Deployment, but pods get stable, numbered names (`-0` , `-1` , `-2`) that survive restarts. That's what makes the DNS names above predictable enough to hard-code into the router.

- `podManagementPolicy: Parallel` starts all replicas at once instead of one by one.
- `schedulerName: hami-scheduler` hands placement to HAMi, which enforces the slice.
- The `bp1-config` label records which config (A/B/C) the pod belongs to.

The container. It uses the locally built image (`imagePullPolicy: Never` , since it only exists in k3s's own image store) and runs `vllm-entrypoint.sh` from the `vllm-entrypoint` ConfigMap. The script turns the environment variables into `vllm serve` flags:

Env var	Becomes
<code>MODEL_ID</code> , <code>SERVED_MODEL_NAME</code>	the model and the name clients use (<code>qwen7b</code>)
<code>QUANTIZATION</code>	<code>--quantization fp8</code>
<code>GPU_MEM_UTIL</code> , <code>MAX_NUM_SEQS</code> , <code>MAX_MODEL_LEN</code>	memory fraction of the slice, batch size, context length
<code>ENABLE_PREFIX_CACHING</code>	<code>--enable-prefix-caching</code> or <code>--no-enable-prefix-caching</code>
<code>KV_TRANSFER_CONFIG</code>	if set, <code>--kv-transfer-config</code> with <code>LMCacheConnectorV1</code> ; the script also writes an LMCache config pointing at the Mooncake master
<code>POD_IP</code>	the pod's own IP, used as its Mooncake address

Resources. The GPU request is where HAMi comes in:

- `nvidia.com/gpu: 1` , one virtual GPU.

- `nvidia.com/gpumem: ${GPUMEM}` , the HBM cap in MB. vLLM sees only this much, so `-gpu-memory-utilization 0.9` means 90% of the slice.
- `nvidia.com/gpucores: ${GPUCORES}` , the percentage of compute.
- CPU memory: 8 Gi requested, 24 Gi limit (weights are loaded through CPU RAM, and LMCache's CPU tier lives here).

Probes. vLLM takes minutes to start (load weights, compile, capture CUDA graphs), so the probes are staged:

- `startupProbe` : polls `/health` every 10 s for up to 15 minutes. The other probes wait until it passes.
- `readinessProbe` : every 5 s; only ready pods get traffic.
- `livenessProbe` : every 20 s; six failures in a row (2 minutes) restarts the container.

Volumes.

- `hf-cache` → the downloaded weights.
- `vllm-cache` → `/home/ubuntu/vllm-cache` , where `torch.compile` results are kept so restarts are faster.
- `shm` → a 4 Gi in-memory `/dev/shm` . vLLM's worker processes share data through it, and Docker's 64 MB default is too small.

Routers: one Service, two implementations

Only one router runs at a time, and both sit behind the same `router` Service. That way LiteLLM and the benchmarks always call `router:8080` and never need to know which config is active. Config A has no router at all.

```
router/router-service.yaml
```

A ClusterIP **Service** named `router` on port 8080. Its selector is `tier: router` , a label that both router Deployments carry. Whichever router is deployed receives the traffic.

```
router/sglang-router.yaml (config B)
```

A **Deployment** running SGLang's router with `--policy cache_aware` : requests that start with the same text go to the same decode pod, where vLLM already has that prefix cached in HBM.

- It uses a plain `python:3.12-slim` image and runs `pip install sglang-router==0.3.2` at startup, then launches `sglang_router.launch_router`.
- `--worker-urls` lists each decode pod by its StatefulSet DNS name (`vllm-decode-0.vllm-decode...:8000` , and so on). That's why the vLLM pools use headless Services.
- Prometheus metrics are on port 29000, named `metrics` so the `routers` PodMonitor finds them.
- `readinessProbe` on `/health` allows up to 5 minutes (60×5 s), because the pip install happens first.

`router/kv-router.yaml` (config C, or config B with `ROUTER_IMPL_B=kv`)

A **Deployment** running this repo's `router/kv_router.py` , mounted from the `kv-router-code` ConfigMap. It uses the app image, which already has `aiohttp` , so nothing is installed at startup.

It's configured entirely through env vars:

Env var	Value	Meaning
<code>DECODE_URLS</code>	the 3 decode pod URLs	Where the answer is generated
<code>PREFILL_URLS</code>	the 2 prefill pod URLs (empty in config B)	If set, each request first goes to a prefill pod
<code>POLICY</code>	<code>cache_aware</code>	Longest-matching-prefix wins, unless the pods are unevenly loaded
<code>PORT</code> / <code>METRICS_PORT</code>	8080 / 9100	Traffic and Prometheus metrics
<code>NVIDIA_VISIBLE_DEVICES</code>	<code>void</code>	Keeps this CPU-only pod off the GPU

For each request in config C it:

1. Picks a prefill pod and sends it a copy of the request with `max_tokens=1` . The prefill pod processes the prompt, and LMCache writes the resulting KV cache to Mooncake.
2. Picks a decode pod by prefix match and forwards the real request, streaming the tokens back. The decode pod finds the prompt's KV in its HBM cache or in Mooncake, so it skips most of the prompt processing.

If the prefill step fails, it still sends the request to decode, which then processes the prompt itself.

Gateway and bench client

The gateway is the only part of the stack reachable from outside the node; the bench client is the in-cluster machine that generates load.

`gateway/litellm.yaml`

Four objects that put LiteLLM in front of the router:

Kind	Name	What it holds
Secret	<code>litellm-master-key</code>	The API key clients must send (randomly generated into <code>.secrets.env</code> by phase 7)
ConfigMap	<code>litellm-config</code>	LiteLLM's <code>config.yaml</code>
Deployment	<code>litellm</code>	The LiteLLM server on port 4000
Service	<code>litellm</code>	<code>NodePort</code> 30400, so the node's own IP answers on port 30400

What the config says:

- **One model, `qwen7b`**, routed as `hosted_vllm/qwen7b` to `http://router.inference.svc.cluster.local:8080/v1`. The `api_key: dummy` is only because the provider expects one; the router doesn't check it.
- **`cache: false`**. LiteLLM can cache whole responses; left on, repeated benchmark prompts would be answered without touching vLLM.
- **`master_key: os.environ/LITELLM_MASTER_KEY`**. The key is read from an env var filled from the Secret, so it never appears in the ConfigMap. Requests without `Authorization: Bearer <key>` get 401.
- **Timeouts of 600 s**, so long generations under heavy load aren't cut off.

The Deployment runs 2 worker processes (`--num_workers=2`) and becomes ready once `/health/liveliness` answers. NodePort means "open this port on the node itself"; it's what makes `http://<LAMBDA_IP>:30400` work, if your Lambda firewall allows it.

`bench/bench-client.yaml`

A **Deployment** whose pod just runs `sleep infinity`. The benchmark script uses `kubectl exec` to run commands inside it. Running the load generator inside the cluster lets it reach each vLLM pod by DNS name and keeps laptop-to-node network latency out of the measurements.

- It uses the app image, so `vllm bench serve` and `aiohttp` are already installed.
- `/bench` → `/home/ubuntu/bench` on the node. Results written here survive pod restarts and are what `laptop.sh fetch` copies back.
- `/scripts` → the `bench-scripts` ConfigMap, built from the repo's `bench/` folder (`multiturn_bench.py`, `scrape_metrics.py`, `analyze.py`).
- The Hugging Face cache is mounted too, so `vllm bench` can load the Qwen tokenizer without downloading it again.
- It asks for no GPU. It gets up to 6 CPUs and 16 Gi of memory so the client isn't the bottleneck at `rate=inf`.

Cheat sheet: looking at these objects on the node

Run these on the node to see what each file actually created.

To see	Command
Everything in the stack	<code>kubectl get all -n inference</code> and <code>kubectl get all -n kv-tier</code>
A template after rendering, without applying it	<code>source scripts/lib.sh && render manifests/gateway/litellm.yaml</code>
Which config is active	<code>cat ~/blueprint1/.active_config</code>
Per-pod DNS names behind a pool	<code>kubectl -n inference get endpoints vllm-decode -o wide</code>
A pod's GPU slice	<code>kubectl -n inference get pod vllm-decode-0 -o jsonpath='{.spec.containers[0].resources.limits}'</code>
The <code>vllm serve</code> command and LMCache config a pod used	<code>kubectl -n inference logs vllm-decode-0 head -40</code>

To see	Command
Which pod the router is sending to	<code>kubect1 -n inference exec deploy/bench-client -- curl -s router:8080/workers (kv-router only)</code>
Whether Prometheus scrapes a PodMonitor	Prometheus UI → Status → Targets (via <code>laptop.sh tunnel</code> , port 9090)
Mooncake master activity	<code>kubect1 -n kv-tier logs deploy/mooncake-master --tail=50</code>
Why a pod is Pending	<code>kubect1 -n inference describe pod <name></code> (look at Events; HAMi explains slice shortages there)
All layers at once	<code>./scripts/status.sh</code>